

Prueba Práctica de Automatización

Rol: QA Automation Engineer — automatización de UI y API con Playwright, Cucumber y Allure

PLAZO 48 o 72 horas	CAPAS UI + API	SISTEMA BAJO PRUEBA Portal DTE (CRUD)	ENTREGA Repositorio Git
------------------------	-------------------	--	----------------------------

1. Qué se te pide

Esta prueba valida que sepas automatizar un flujo real en **dos capas —UI y API—**, con escenarios escritos en **Cucumber (Gherkin)**, pruebas ordenadas, estables y con un reporte **Allure** claro.

El sistema bajo prueba es un portal de documentos tributarios electrónicos (DTE) con un **CRUD completo**. Debes automatizar **el mismo ejercicio en ambas capas**: crear, leer, editar y eliminar un documento tributario, comprobando cada resultado con aserciones (incluida la comprobación de que lo eliminado *ya no existe*).

El contrato de la API, las reglas de validación y el ciclo de vida de los documentos están resumidos en el [Anexo](#) y detallados en el `README.md` del laboratorio. Léelos antes de empezar.

2. Stack obligatorio

Componente	Requisito
Framework	Playwright.
Lenguaje	TypeScript con tipado estricto, sin <code>any</code> .
Enfoque BDD	Cucumber / Gherkin (<code>@cucumber/cucumber</code>). Puedes ejecutarlo con <code>cucumber-js</code> o integrarlo con Playwright Test mediante <code>playwright-bdd</code> .
Reportabilidad	Allure Report: <code>allure-cucumberjs</code> si usas <code>cucumber-js</code> , o <code>allure-playwright</code> si usas <code>playwright-bdd</code> .
Diseño	Page Object Model (POM) para la UI y cliente API modular para el backend.

3. Entorno y accesos

El laboratorio corre en tu máquina con Docker (aplicación Next.js + PostgreSQL). Requisito: Docker Desktop con Docker Compose.

```
git clone <URL del repositorio que te indique el facilitador>
cd azurian-qa-evaluation
docker compose up --build
```

Para detenerlo: `docker compose down`. Para dejar los datos locales como nuevos: `docker compose down -v`.

Dato	Valor
URL del portal	<code>http://localhost:3000</code> (o la que indique el facilitador)

Dato	Valor
Base URL de la API	http://localhost:3000/api
Usuario / contraseña	admin@azurian.com / Azurian2026!
Verificar que está listo	GET /api/health responde 200

Puerto en Windows. El sistema puede reservar el puerto 3000 y Docker fallará con *"forbidden by its access permissions"*. En ese caso publica el portal en otro puerto con la variable `APP_PORT` (por ejemplo, copia `.env.example` a `.env` : trae `APP_PORT=3100`) y usa esa URL. Tu suite debe leer la URL base desde una variable de entorno, no dejarla fija en los escenarios.

4. Reglas del reto

Aislamiento por ejecución. Usa tu propio `runId` (3–80 caracteres alfanuméricos, `_` o `-`) en el encabezado `x-lab-run-id` . Cada `runId` parte de **48 documentos semilla**. Tus escenarios no deben chocar con los de otra ejecución ni depender de un orden: genera folios y RUT únicos por escenario. Para los flujos que mezclan API y UI, establece ese mismo valor en `localStorage` con la clave `azurian_lab_run_id` antes de abrir el portal.

Latencia. El portal responde con **latencia aleatoria de 0,3 a 3,2 segundos**. Tu suite debe ser estable frente a esa variabilidad.

Permitido y esperado

- `getByRole` , `getByLabel` , `getByText` y relaciones semánticas de tabla.
- `expect` , `auto-wait` de Playwright y esperas ligadas a un evento observable (respuesta, indicador de carga, diálogo, mensaje).
- POM, cliente API tipado y datos únicos por escenario.

Se evalúa negativamente

- XPath absoluto, clases de estilo, IDs internos o posiciones globales (`first()` , `nth()`) para elegir un documento.
- `waitForTimeout` , `sleep` o cualquier pausa fija.
- Leer o modificar PostgreSQL, las latencias o el seed, o usar datos de otra ejecución.
- Dar por buena una prueba porque "la primera fila coincide": valida la fila exacta por folio, RUT y monto.

5. Escenarios obligatorios

Escribe los escenarios en archivos `.feature` (Gherkin) y automatiza **el mismo ejercicio en las dos capas**: el ciclo de vida de un documento tributario. Cada operación debe tener aserciones que comprueben el resultado.

Operación	En la UI (<code>@ui</code>)	En la API (<code>@api</code>)
Crear	Inicia sesión, abre "Nuevo documento", completa el formulario y emite. Verifica el aviso de éxito y que la fila muestra folio, tipo, RUT, monto y estado.	POST <code>/api/documents</code> con sesión válida. Verifica <code>201</code> y el cuerpo completo de la respuesta (datos enviados, <code>id</code> y <code>estado</code> asignados, fechas), y que GET <code>/api/documents/{id}</code> devuelve exactamente lo mismo.
Leer	Filtra por RUT, identifica la fila exacta por su folio (el RUT tiene varios documentos) y abre "Ver detalle". Verifica los datos del detalle.	Lista con filtros (<code>rut + tipoDte</code>), localiza el folio exacto y lo lee por id. Verifica el cuerpo completo del listado y del detalle; un id inexistente responde <code>404</code> con su error completo.
Editar	Abre la edición de un documento propio, cambia datos y guarda. Verifica el aviso, la fila actualizada y que el cambio persiste al recargar.	PUT y/o PATCH sobre un documento propio. Verifica <code>200</code> y el cuerpo completo (campos cambiados y los que no deben cambiar), y que un GET posterior lo confirma.
Eliminar	Elimina un documento propio pasando por el diálogo de confirmación. Verifica que ya no existe : la fila desaparece, la tabla queda vacía para ese filtro y sigue así al recargar.	DELETE sobre un documento propio. Verifica <code>204</code> sin cuerpo y que ya no existe : GET <code>/api/documents/{id}</code> responde <code>404</code> con su error completo y no aparece en el listado.

Cómo validar la API

No basta con revisar algunos campos: cada respuesta debe compararse por su **cuerpo completo** (todos los campos, ninguno de más ni de menos). Los valores que cambian en cada ejecución (`id`, fechas, token) se validan por su forma en lugar de por su valor.

Cómo debe explicarse el reporte

Cada verificación de tu reporte Allure debe dejar claro **qué se esperaba y qué se obtuvo** (por ejemplo, adjuntando el cuerpo esperado y el recibido), para que quien lo lea entienda por qué la prueba está bien sin abrir el código.

Cómo escribir el Gherkin

Los escenarios describen el comportamiento de negocio (*qué ocurre*), no la implementación (*cómo*): sin selectores, URLs ni rutas de API dentro del `.feature`. Reutiliza las mismas frases en varios escenarios, usa `Antecedentes` para lo común, tablas de datos para los datos de negocio y `Esquema del escenario` con `Ejemplos` para los casos repetidos.

6. Extras que suman puntos (opcionales)

- **Validaciones de API:** campo obligatorio ausente, RUT con dígito verificador inválido, folio duplicado (`409`), petición sin token (`401`), con el `code` y el `field` esperados.
- **Validaciones de UI:** errores del formulario junto a cada campo, correo de contacto habilitado solo cuando corresponde, acciones bloqueadas de un documento aceptado.
- **Flujo híbrido:** crear por API y verificar en pantalla (o al revés).
- **Buenas prácticas adicionales:** casos parametrizados, fixtures propios, paralelismo seguro, manejo de la vigencia del token.

7. Orden del código

Cada escenario debe poder **identificarse y ejecutarse por separado**. Se espera:

- Un **título único y descriptivo**, idealmente con un ID (`UI-CRUD-01 · Crear un documento tributario`).
- Los `.feature` , las definiciones de los pasos, las páginas y los clientes en carpetas separadas por capa (`api/` y `ui/`).
- **Tags** que permitan ejecutar una capa (`@api` , `@ui`) o una operación (`@crear` , `@leer` , `@editar` , `@eliminar`).
- Definiciones de pasos delgadas: cada frase solo llama a un Page Object, a un cliente API o a un verificador.
- Configuración (URL, credenciales) centralizada, no repartida entre los escenarios.

Estructura de proyecto sugerida

```
tu-solucion/
├── api/
│   ├── features/      # escenarios Gherkin de la API (*.feature)
│   ├── steps/         # definición de cada frase
│   ├── clients/       # cliente API modular
│   ├── models/        # tipos de las respuestas
│   └── fixtures/
├── ui/
│   ├── features/      # escenarios Gherkin de la UI (*.feature)
│   ├── steps/
│   ├── pages/         # Page Objects
│   └── fixtures/
├── shared/            # configuración y datos de prueba
├── playwright.config.ts # (o cucumber.js si usas cucumber-js)
├── tsconfig.json
├── package.json       # scripts (atajos)
└── README.md
```

8. Instrucciones de entrega

1. Sube tu solución a un repositorio público o privado en **GitHub / GitLab** (si es privado, otorga acceso a la cuenta indicada por el facilitador).
2. El `README.md` de tu solución debe incluir los comandos exactos para:
 - Instalar dependencias y configurar el entorno.
 - Ejecutar la suite completa.
 - Ejecutar **solo API** o **solo UI**.
 - Ejecutar por **tag** o un escenario individual.
 - **Generar y abrir el reporte Allure** localmente.
3. Agrega scripts de `npm` como atajos para lo anterior.
4. Envía el enlace de tu repositorio antes de la fecha límite respondiendo al correo de la convocatoria.

9. Cómo se evalúa

La evaluación es **acumulativa**: no existen requisitos eliminatorios. Cada incumplimiento solo resta los puntos del indicador correspondiente. Tu puntaje final (sobre 100, más hasta 10 puntos extra) resulta de:

Criterio	Puntos
1. Cobertura funcional del CRUD (crear, leer, editar y eliminar en UI y API)	16
2. Calidad de las aserciones (datos de negocio, cuerpo completo de la API, persistencia y comprobación de lo eliminado)	16
3. Localizadores de la UI (semánticos, fila exacta, sin selectores frágiles)	10
4. Manejo de asincronía y estabilidad (sin pausas fijas, esperas observables, suite estable)	10
5. Datos de prueba e independencia (datos únicos, escenarios independientes)	6
6. Arquitectura y organización del código (POM, cliente API, identificación de cada escenario)	12
7. BDD con Cucumber / Gherkin (Gherkin declarativo, frases reutilizables, recursos de Gherkin, escenarios atómicos)	10
8. TypeScript y calidad de código (estricto, sin <code>any</code> , tipos)	6
9. Allure y evidencias (reporte configurado, legible y explicativo —lo esperado frente a lo obtenido—, con evidencia de fallos y de las llamadas API)	10
10. Documentación y ejecución (README, ejecución por tags, scripts)	4
Total base	100
Puntos extra (validaciones de API y UI, flujo híbrido, buenas prácticas)	hasta +10

Niveles: 90–100 Sobresaliente · 70–89 Aprobado · 50–69 Por reforzar · 0–49 No alcanza el nivel esperado.

Cucumber y Allure son obligatorios. Sin Cucumber, el criterio 7 queda en 0 puntos; sin Allure, el criterio 9 queda en 0 puntos. En ninguno de los dos casos hay otra consecuencia.

Anexo · Contrato del laboratorio

Resumen de referencia. El detalle completo está en el `README.md` del repositorio del laboratorio.

Autenticación

`POST /api/auth/login` con `{ "email": "admin@azurian.com", "password": "Azurian2026!" }` devuelve `200` con un JWT (HS256), su vigencia en segundos y el usuario:

```
{ "token": "eyJhbGciOi...", "expiresIn": 3600,
  "user": { "id": "user-azurian-01", "email": "admin@azurian.com",
            "name": "Administrador Azurian", "role": "QA Lab Participant" } }
```

Las rutas de documentos y `/api/auth/me` exigen `Authorization: Bearer <token>` y el encabezado `x-lab-run-id`. Un token ausente, alterado o vencido responde `401` (`UNAUTHORIZED` o `TOKEN_EXPIRED`).

Endpoints

Método	Ruta	Resultado
POST	<code>/api/auth/login</code>	<code>200</code> con token · <code>400</code> datos faltantes · <code>401</code> credenciales inválidas.
GET	<code>/api/auth/me</code>	<code>200</code> con el usuario y <code>expiresAt</code> · <code>401</code> .
GET	<code>/api/documents</code>	<code>200</code> con un arreglo de documentos (filtros y paginación opcionales).
POST	<code>/api/documents</code>	<code>201</code> con el documento creado (<code>PENDIENTE</code>) · <code>400</code> · <code>409</code> folio duplicado.
GET	<code>/api/documents/{id}</code>	<code>200</code> · <code>404</code> .
PUT	<code>/api/documents/{id}</code>	<code>200</code> reemplazo de los campos editables · <code>400</code> · <code>404</code> · <code>409</code> documento aceptado.
PATCH	<code>/api/documents/{id}</code>	<code>200</code> actualización parcial · <code>400</code> · <code>404</code> · <code>409</code> .
DELETE	<code>/api/documents/{id}</code>	<code>204</code> sin cuerpo · <code>404</code> · <code>409</code> documento aceptado.
POST	<code>/api/documents/{id}/send</code>	<code>200</code> con el documento ya resuelto por el SII (simulado) · <code>404</code> · <code>409</code> si no está <code>PENDIENTE</code> .
GET	<code>/api/documents/summary</code>	<code>200</code> con totales de la ejecución.
GET	<code>/api/health</code>	Healthcheck del laboratorio.

Listado, filtros y paginación

`GET /api/documents?rut=&tipoDte=&estado=&page=&pageSize=`

Parámetro	Valores	Notas
<code>rut</code>	dígitos, puntos, guion y <code>K</code> (máx. 12)	Búsqueda parcial que ignora puntos y guion: <code>76192584</code> encuentra <code>76.192.584-9</code> .
<code>tipoDte</code>	<code>DTE 33</code> , <code>DTE 34</code> , <code>DTE 39</code> o <code>TODOS</code>	
<code>estado</code>	<code>ACEPTADO</code> , <code>PENDIENTE</code> , <code>RECHAZADO</code> o <code>TODOS</code>	
<code>page</code> , <code>pageSize</code>	enteros ≥ 1 y $1-100$	Si se envía alguno, se pagina (por defecto página 1 de 10). Sin ellos se devuelve todo.

El cuerpo es siempre un arreglo. Los metadatos van en encabezados: `x-total-count` (siempre) y, al paginar, `x-page`, `x-page-size` y `x-total-pages`. Orden: `fechaEmision` descendente y luego `folio` descendente.

Modelo de documento

```
{ "id": "1f0c...", "tipoDte": "DTE 33", "folio": 9901,
  "rutReceptor": "76.543.209-K", "monto": 450000, "fechaEmision": "2026-09-28",
  "estado": "PENDIENTE", "deliveryChannel": "EMAIL", "sendCopy": true,
  "contactEmail": "contacto@empresa.cl", "observaciones": null,
  "attachmentName": "respaldo.pdf",
  "createdAt": "2026-09-28T22:44:39.000Z", "updatedAt": "2026-09-28T22:44:39.000Z" }
```

`estado`, `id`, `createdAt` y `updatedAt` los asigna el servidor: si se envían, se ignoran (igual que cualquier campo desconocido).

Reglas de validación

Se aplican en el servidor y también en el formulario (mismas reglas, mismos mensajes). La API valida tipos de forma estricta (`"123"` no es un número) y devuelve **todos** los errores a la vez.

Campo	Requerido	Regla
<code>tipoDte</code>	sí	DTE 33, DTE 34 o DTE 39.
<code>folio</code>	sí	Entero de 1 a 9.999.999. Único por ejecución.
<code>rutReceptor</code>	sí	Formato 12.345.678-9 (con puntos y guion) y dígito verificador válido (módulo 11) . Una <code>k</code> minúscula se normaliza a <code>K</code> .
<code>monto</code>	sí	Entero en CLP de 1 a 999.999.999.
<code>fechaEmision</code>	sí	YYYY-MM-DD, fecha real de calendario, entre 2000-01-01 y hoy.
<code>deliveryChannel</code>	no (PORTAL)	PORTAL o EMAIL.
<code>sendCopy</code>	no (false)	Booleano.
<code>contactEmail</code>	condicional	Obligatorio si <code>sendCopy</code> es <code>true</code> o el canal es EMAIL . Si se envía, debe ser un correo válido (máx. 120).
<code>observaciones</code>	no	Texto de hasta 300 caracteres.
<code>attachmentName</code>	no	Nombre con extensión <code>.pdf</code> , <code>.xml</code> o <code>.txt</code> (máx. 120, sin <code>/</code> ni <code>\</code>). En la UI el archivo no puede superar 2 MB.

`PUT` es un **reemplazo**: exige los campos obligatorios y los opcionales omitidos vuelven a su valor por defecto. `PATCH` valida solo lo enviado (combinado con el estado actual) y rechaza un cuerpo sin campos editables.

Ciclo de vida y reglas de estado

Estado	Editar (<code>PUT</code> / <code>PATCH</code>)	Eliminar	Enviar al SII
PENDIENTE	Sí (sigue PENDIENTE)	Sí	Sí
RECHAZADO	Sí, vuelve a PENDIENTE	Sí	No
ACEPTADO	No → 409 DOCUMENT_LOCKED	No → 409 DOCUMENT_LOCKED	No

- Todo documento nuevo nace `PENDIENTE`.

- `tipoDte` y `folio` son **inmutables**: enviarlos con otro valor devuelve `400` (con el mismo valor está permitido).
- **Enviar al SII** es una simulación determinista: `monto` $\leq 10.000.000$ → `ACEPTADO`; mayor → `RECHAZADO`. No se conecta a ningún sistema real.

Formato de error

```
{ "error": "Datos de entrada inválidos. Revisa los campos indicados.",
  "code": "VALIDATION_ERROR",
  "details": [{ "field": "monto", "message": "El monto es obligatorio." }] }
```

HTTP	code
400	VALIDATION_ERROR (con details), INVALID_JSON, INVALID_RUN_ID
401	UNAUTHORIZED, TOKEN_EXPIRED, INVALID_CREDENTIALS
404	NOT_FOUND
409	DUPLICATE_FOLIO, DOCUMENT_LOCKED, INVALID_STATE

Orden de precedencia: `401` → `400` (`x-lab-run-id`) → `404` (documento inexistente) → `400` (cuerpo) → `409` (estado).

Datos semilla de cada ejecución

Dimensión	Valores
Documentos	48 por <code>runId</code> , sembrados la primera vez que se usa.
Tipo	16 × <code>DTE 33</code> , 16 × <code>DTE 34</code> , 16 × <code>DTE 39</code>
Estado	17 × <code>ACEPTADO</code> , 16 × <code>PENDIENTE</code> , 15 × <code>RECHAZADO</code>
Monto total	\$ 36.546.400
Receptores	15 RUT; tres tienen 4 documentos y el resto 3. Un RUT puede tener varios documentos del mismo tipo a propósito: acota por filtros y valida la fila exacta por folio, monto u otro dato de negocio.

Interfaz

- **Login** con validación en línea y error de credenciales.
- **Dashboard**: saludo al usuario, tarjetas de resumen, tabla con filtros por RUT, tipo y estado, y paginación de 10 filas.
- **Crear** y **editar** en un diálogo con distintos tipos de campo (select, fecha, número, texto, radio, checkbox, correo, archivo y área de texto). En edición, tipo y folio quedan bloqueados.
- **Ver detalle**, **Enviar al SII** y **Eliminar** (con diálogo de confirmación). Las acciones no aplicables aparecen deshabilitadas.
- Avisos de resultado (`role="status"`) y errores (`role="alert"`). El portal está rotulado semánticamente; los identificadores internos no son contrato de automatización.